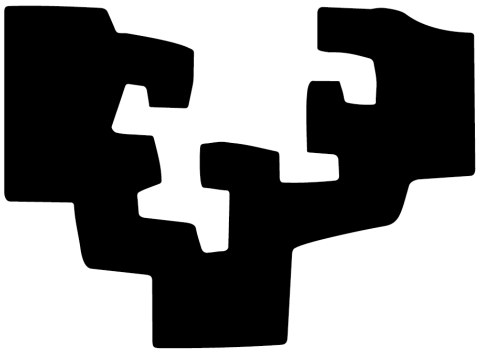
Activity 1.2. A network file system

Introduction **2**
Description of the SZA/SAR application **2**
Users and Authentification 2
File Management and Storage 2
Protocol Syntax and Semantics 2
Implementation Details 2
Comparison of properties of distributed files services and SZA/SAR **3**
Properties that match 3
Properties that match partially 3
Properties that do not match 4
Conclusion **4**
References **4**



Introduction

This report explains the SZA/SAR file transfer service studied in the Distributed Systems course 2026/27 at EHU. It is a simple client-server application built over TCP that lets users list, download, upload and delete files on a server.
The application will be explained and compared to the properties of distributed file services and the cloud file service “Google Drive”.

Description of the SZA/SAR application

The SZA/SAR application is a network-based file service that enables users to list, download, upload, and delete files on a server. It operates over TCP and utilizes a custom text-based application layer protocol for communication between the client and the server.

Users and Authentification

- The application supports authenticated users who must provide a username and a password to log in.
- Authenticated users (“sar” and “sza”) have full permission to upload, download and delete files.
- A special “anonimous” user is available for public access. This user accepts any password but is restricted to downloading files and cannot perform updates or deletions.

File Management and Storage

- Files are stored in the servers working directory, and the protocol does not support folder management or subdirectories.
- The server enforces a maximum file size limit of 10 MiB for uploads.
- A storage space margin of 50 MiB is maintained to prevent the server's disk from reaching full capacity.

Protocol Syntax and Semantics

- Client commands consist of 4-character ASCII strings (USER, PASS, LIST, DOWN, UPLO, DELE, EXIT).
- The server responds with 2-character ASCII codes: “OK” for successful operations “ER” for errors, which are followed by specific parameters or error codes.

Implementation Details

- The application is implemented in Python, with the server listening on port 6012 by default.
- The client script provides an interactive command line menu for the user to execute file operations.

- The server architecture includes implementations for handling multiple concurrent clients, using either multithreading (Multithread Server file) or multi processing (Server File).

Comparison of properties of distributed files services and SZA/SAR

This section compares the properties of a distributed file system that SAR matches, fully, partly or not at all, with Google Drive as a reference cloud Service.
For each section it explains, how far SAR fulfills it and how Google Drive realises it.

Properties that match

- **Permanent information storage:** Files must be saved permanently.
    - SAR: Uploaded files are written to the server folder and remain there, until they are deleted.
    - Drive: Files are stored persistently and redundantly in Google's data centres.
- **Concurrent access from several processes:** Several clients can use the same resource at the same time.
    - SAR: The server starts one process thread per connection, so several clients can list, download and upload at once. However files can’t be locked so when several clients interact with the same file at the same time issues occur. For Example if two clients upload the same file at the same time one upload can get lost completely, without any error messages.
    - Drive: Real-time co-editing of Google documents, and version history lets users recover overwritten changes.
- **Access protection:** Different users have different rights.
    - SAR: Login with user and password. The anonymous user can only download. There are no rights per file, all users share one folder.
    - Drive: has sharing permissions (viewer, commenter, editor) per file or folder for users, groups or links.

Properties that match partially

- **Namespace identifying files:**
    - SAR: Files are identified by name, satisfying the base requirement. However, the namespace is a single directory with no hierarchy, so the structured part is missing (outside of multithread).
    - Drive: Users can organise files in folders, both in their own drive and in shared drives.
- **Robustness to failures:** A client crashing should not damage the server and repeating a request after a failure should be safe.
    - SAR: When a client crashes, only its own process is affected and the server is fine. However resending a DOW2 command after a failure results in an error.
    - Drive: Client disconnects and retries are transparent. Repeating requests after a dropped connection does not result in errors.

- **Consistency:** Users should see a coherent, up-to-date view of the files.
    - SAR:There's no caching, but there is also nothing stopping two people from overwriting or deleting the same file at the same time.
    - Drive: Synchronization keeps changes from multiple devices consistent
- **Security:** Remote user authentication based on credentials.
    - SAR: Authentication by user and password per session, sent in plain text. Content is unencrypted.
    - Drive: Google account authentication with two-factor login and encrypted connections.

Properties that do not match

- **High availability and fault tolerance**
    - SAR: Each file exists in exactly one copy, on one server. The files are only available if the server is running. There is no disconnected or offline mode.
    - Drive: You can access the Drive from anywhere at any time. There is also an offline mode with local synchronization
- **Mobility**: Information is separated from the device.
    - SAR: The user must manually upload or download on every device, and there is no local copy.
    - Drive: Every change is updated automatically across devices.
- **Scalability**
    - SAR: Single, localized server managing a local file directory. No way to add more servers with better hardware.
    - Drive: Google Drive uses multiple resources from different data centers to automatically scale the required resources

Conclusion

To sum up, the SAR/SZA application does not meet the requirements of a distributed system. It lacks properties in relation to a cloud system like security, synchronization and high availability.

References

Distributed Systems 2026/27: Topic1-notes
Distributed Systems 2026/27: Topic4-notes
SZA/SAR Source Code and Documentation